Real-world (rebuild Project 2 as a graph) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `agent_loop_to_graph_practice.py`: this is where Doc07's whole ReAct loop gets rebuilt as a graph — the same "think, act, observe, repeat" shape, with nodes and edges instead of a `for` loop, and Doc07's own `AGENT_TOOLS` and `run_tool()` doing the real work. **If not:** the Build Task's graph would be the first place you ever turned a loop into a graph, with no smaller version to trust it against.

Before you start: copy Doc07's Build Task files — `agent.py`, `docs_search.py`, `exceptions.py`, `http_client.py`, `logging_setup.py`, `tools.py` — into `09_langgraph/practice/`, next to this file, unchanged. Then `from agent import AGENT_TOOLS, run_tool` works.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — the direct translation

```
# agent_loop_to_graph_practice.py
import operator
from typing import Annotated, TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from agent import AGENT_TOOLS, run_tool


class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    steps: Annotated[list, operator.add]


def think(state):
```

```
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools(AGENT_TOOLS).invoke(state["messages"])
    return {"messages": [response]}


def act(state):
    tool_messages = []
    steps = []
    for call in state["messages"][-1].tool_calls:
        result, error = run_tool(call, AGENT_TOOLS)
        observation = result
        if error is not None:
            observation = "Error: " + error
        tool_messages.append({
            "role": "tool",
            "content": observation,
            "tool_call_id": call["id"],
        })
        steps.append({"tool": call["name"], "result": result, "error": error})
    return {"messages": tool_messages, "steps": steps}


def route(state):
    if state["messages"][-1].tool_calls:
        return "act"
    return "finish"


builder = StateGraph(AgentState)
builder.add_node("think", think)
builder.add_node("act", act)
builder.add_edge(START, "think")
builder.add_conditional_edges("think", route, {"act": "act", "finish": END})
builder.add_edge("act", "think")
graph = builder.compile()

task = {"messages": [{"role": "user", "content": "What is 12 + 30?"}]}
result = graph.invoke(task)
print("answer:", result["messages"][-1].content)
for step in result["steps"]:
    print("  step:", step["tool"], "->", step["result"])
```

Expected output (the wording varies; the tool and its result don't):

```
answer: 12 + 30 is 42.
  step: add -> 42
```

- **The state** holds Doc07's `messages` list and the step log. `Annotated[list, operator.add]` means each node's returned list is **added** to the old one — just like Doc07's `messages.append(...)`.
- **`think`** is Doc07's model call: `bind_tools(AGENT_TOOLS)`, then `.invoke(messages)`. It returns the reply, which gets added to `messages`.
- **`act`** is the inner `for call in response.tool_calls:` loop from Doc07, using the same `run_tool()`, and the same `{"role": "tool", ...}` messages.
- **`route`** is Doc07's `if not response.tool_calls: return ...` check, turned into an edge: tool calls → `act`, none → `END`.
- **`act → think`** is the loop's "go round again".

This version works correctly. It's missing type hints and a step limit — both fine for a first working version.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, a `build_graph()` function, and a step limit

```
# agent_loop_to_graph_practice.py
import operator
from typing import Annotated, TypedDict

from langchain_openai import ChatOpenAI
from langgraph.graph import END, START, StateGraph

from agent import AGENT_TOOLS, run_tool

RECURSION_LIMIT = 20


class AgentState(TypedDict):
    messages: Annotated[list, operator.add]
    steps: Annotated[list[dict], operator.add]


def think(state: AgentState) -> dict:
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    response = model.bind_tools(AGENT_TOOLS).invoke(state["messages"])
    return {"messages": [response]}


def act(state: AgentState) -> dict:
    tool_messages: list[dict] = []
    steps: list[dict] = []
    for call in state["messages"][-1].tool_calls:
        result, error = run_tool(call, AGENT_TOOLS)
        observation = result
        if error is not None:
            observation = f"Error: {error}"
        tool_messages.append({
            "role": "tool",
            "content": observation,
            "tool_call_id": call["id"],
        })
        steps.append({"tool": call["name"], "result": result, "error": error})
```

```
    return {"messages": tool_messages, "steps": steps}


def route(state: AgentState) -> str:
    if state["messages"][-1].tool_calls:
        return "act"
    return "finish"


def build_graph():
    builder = StateGraph(AgentState)
    builder.add_node("think", think)
    builder.add_node("act", act)
    builder.add_edge(START, "think")
    builder.add_conditional_edges(
        "think", route, {"act": "act", "finish": END}
    )
    builder.add_edge("act", "think")
    return builder.compile()


def run_graph(graph, task: str) -> dict:
    start = {"messages": [{"role": "user", "content": task}], "steps": []}
    # why: the graph's version of Doc07's max_iterations
    return graph.invoke(start, config={"recursion_limit": RECURSION_LIMIT})


if __name__ == "__main__":
    graph = build_graph()
    result = run_graph(graph, "What is 12 + 30?")
    print(f"answer: {result['messages'][-1].content}")
    for step in result["steps"]:
        print(f"  step: {step['tool']} -> {step['result']}")
```

Expected output:

```
answer: 12 + 30 is 42.
  step: add -> 42
```

Difference from Basic: full type hints, and the graph is built inside `build_graph()` and run by `run_graph()`, so other code (Approach 2) can reuse both. `recursion_limit` is the graph's version of Doc07's `max_iterations`: without it, a model that keeps asking for tools would loop until LangGraph's default limit of 25 steps, instead of the limit you chose.

Approach 2 — a real equivalence check against the original loop

Story: the exercise's own claim is that the graph behaves the *same* as your Doc07 loop — a claim needs a check, not one manual run you eyeballed. Model wording changes from run to run, so compare what doesn't: **which tools each one used**. **If not:** a routing bug that skips a tool but still writes a plausible answer would slip into the Build Task.

```
# agent_loop_to_graph_practice.py — add below Approach 1's code
from agent import run_agent

TEST_PROMPTS = [
    "What is 12 + 30?",
    "What's the weather in Lahore?",
    "What is the capital of France?",
]


def tool_names(steps: list[dict]) -> list[str]:
    names = []
    for step in steps:
        names.append(step["tool"])
    return names


def compare_with_loop(graph) -> None:
    mismatches = []
    for prompt in TEST_PROMPTS:
        loop_tools = tool_names(run_agent(prompt, 5).steps)
        graph_tools = tool_names(run_graph(graph, prompt)["steps"])
        status = "MATCH"
        if loop_tools != graph_tools:
            status = "MISMATCH"
            mismatches.append(prompt)
        print(f"[{status}] {prompt!r}: loop={loop_tools} graph={graph_tools}")
    if mismatches:
        raise AssertionError(f"graph and loop disagreed on: {mismatches}")
    print("All test prompts matched.")


compare_with_loop(build_graph())
```

Expected output:

```
[MATCH] 'What is 12 + 30?': loop=['add'] graph=['add']
[MATCH] "What's the weather in Lahore?": loop=['get_weather']
        graph=['get_weather']
```

```
[MATCH] 'What is the capital of France?': loop=[] graph=[]
All test prompts matched.
```

(Long lines are wrapped here to fit the page.) `run_agent()` is Doc07's real loop, imported unchanged — the comparison is your own two implementations against each other, on the same prompts.

Approach 3 — comparing turns too, not just tools

Story: two runs can use the same tools while one of them takes extra model turns to get there. **If not:** a conditional edge that loops back one time too many would pass Approach 2's check and quietly double the cost.

```
# agent_loop_to_graph_practice.py — add below Approach 2's code
def count_model_turns(messages: list) -> int:
    turns = 0
    for message in messages:
        # how: user and tool messages are dicts; model replies are not
        if not isinstance(message, dict):
            turns = turns + 1
    return turns


graph = build_graph()
for prompt in TEST_PROMPTS:
    result = run_graph(graph, prompt)
    turns = count_model_turns(result["messages"])
    steps = len(result["steps"])
    print(f"{prompt!r}: {turns} model turns, {steps} tool steps")
```

Expected output:

```
'What is 12 + 30?': 2 model turns, 1 tool steps
"What's the weather in Lahore?": 2 model turns, 1 tool steps
'What is the capital of France?': 1 model turns, 0 tool steps
```

A tool task should take 2 model turns (ask for the tool, then answer) and a no-tool task 1. More than that means the graph loops more than it should. `isinstance(message, dict)` tells your own dict messages apart from the model's reply objects — the same check as Doc08b's `isinstance(result, Exception)`.

Difference from Approach 1, and between Approaches 2/3: Approach 1 proves the graph runs on one prompt. Approach 2 backs up "same behavior as my loop" with a check on several prompts —

comparing tools used, because exact wording never matches twice. Approach 3 adds a second signal, model turns, that matching tools alone can hide.

Which one should you actually write? Approach 1's `build_graph()` and step limit, always — it's what the Build Task copies. Run Approach 2's check once against your real Doc07 loop before trusting the rebuild. Add Approach 3 when something about a run's path feels off, or as soon as you have more than a few prompts — it's the cheapest way to catch an edge that loops too often.